Birzeit University

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

ENCS4370: COMPUTER ARCHITECTURE

Project 2

Pipelined RISC Processor

Prepared by:

Name: Carlos Khamashta **Number**: 1220050

Name: Hanna Kaibni **Number**: 1220214

Instructor: Dr. Aziz Qaroush

Section: 1

Date: 10/6/2025

Team Member Name	Team Member ID	Contributions
Hanna Kaibni	1220214	Datapath, Design, Coding, Testing, Documenting, Report
Carlos Khamashta	1220050	Datapath, Design, Coding, Testing, Documenting, Report

Processor Implementation (Tick One)	
Single Cycle	
Multi Cycle	
Pipelined	Yes

In case of pipeline implementation (Tick the correct answer)		
	Implemented in RTL Code?	**Verified and Correctly Worked?**
Data hazards detection	Yes	Yes
Control hazards detection	Yes	Yes
Structural hazards detection	Yes	Yes
Forwarding	Yes	Yes
Stalling	Yes	Yes

Tick the correct answer						
Instruction	**Did you implement this instruction in the RTL?**		**Did you write the verification code for this instruction?**		**Did the instruction Work perfectly when it has been verified?**	
OR Rd, Rs, Rt	Yes		Yes		Yes	
ADD Rd, Rs, Rt	Yes		Yes		Yes	
SUB Rd, Rs, Rt	Yes		Yes		Yes	
CMP Rd, Rs, Rt	Yes		Yes		Yes	
ORI Rd, Rs, Imm	Yes		Yes		Yes	
ADDI Rd, Rs, Imm	Yes		Yes		Yes	
LW Rd, Imm(Rs)	Yes		Yes		Yes	
LDW Rd,Imm(Rs)	Yes		Yes		Yes	
SDW Rd,Imm(Rs)	Yes		Yes		Yes	
BZ Rs, Label	Yes		Yes		Yes	
BGZ Rs, Label	Yes		Yes		Yes	
BLZ Rs, Label	Yes		Yes		Yes	
JR Rs	Yes		Yes		Yes	
J Label	Yes		Yes		Yes	
CALL Label	Yes		Yes		Yes	

Tick one of the following	Yes
My processor can execute only test programs consisting of one instruction only	
My processor can execute complete programs (A simulation screenshot must be provided as evidence)	Yes

Abstract

This report details the development and implementation of a five-stage pipelined RISC(Reduced Instruction Set Computer) processor using Verilog. The processor features a modular architecture, divided into five stages of instruction fetch, decode, execute, memory access, and Write back. This pipelined structure enhances instruction throughput by enabling multiple instructions to be processed simultaneously at different stages. The design supports a comprehensive set of operations, including arithmetic, logical, memory-related, and control flow instructions, along with additional handling for pseudo-instructions. Both the Datapath and control components were carefully designed to maintain synchronization between stages and ensure efficient execution. Comprehensive simulation and testing confirmed the processor's functional correctness and demonstrated its ability to execute instruction sequences accurately and effectively.

Table Of Contents

Abstract III

Introduction to Design and Implementation 1

Development Environment: 1

Design's 32-bit format 1

RTL Design and Instruction Encoding 2

Arithmetic and Logical Register-Based Instructions 2

Arithmetic Immediate Instructions 2

Load Word Instruction (LW) 3

Store Word Instruction (SW) 3

Load Double Word Instruction (LDW) 4

Store Double Word Instruction (SDW) 6

Branch Instructions *(BZ, BGZ, BLZ)* 7

Jump Register Instruction (JR) 8

Jump Instruction (J) 8

Call Instruction (CLL) 9

Datapath 10

The above figure shows the complete Datapath of our Pipelined Processor: 10

1. **Fetch Stage** 10
2. **Decode Stage** 11
3. **Execute Stage** 12
4. **Memory Stage** 12
5. **Write-Back Stage** 13

Processor Simulation and Verification 22

Simulation Test 1: Integer Divider using given instructions 22

Expected Value 22

Simulation Values: 23

Waveform Generation: 23

Simulation Test 2: General Instructions with Stall 24

Expected Value 24

Simulation Values: 25
Waveform Generation: 26
Simulation Test 3: General LDW SDW implementation 26
Expected Value: 27
Simulation Values: 27
Waveform Generation: 28
Teamwork 29
Conclusion 30

Table Of Figures

Figure 1: OR, ADD,SUB, CMP description and Opcode 2
Figure 2:ORI,ADDI description and Opcode 2
Figure 3: LW description and Opcode 3
Figure 4: SW description and Opcode 3
Figure 5: LDW description and Opcode 4
Figure 6: SDW description and Opcode 6
Figure 7: BZ, BGZ, BLZ description and Opcode 7
Figure 8: JR Rs description and Opcode 8
Figure 9: J Label description and Opcode 8
Figure 10: CLL Label description and Opcode 9
Figure 11: Main Datapath Circuit Diagram 10
Figure 12:Fetch Stage Circuit Diagram 10
Figure 13:Decode Stage Circuit Diagram 11
Figure 14:Execute Stage Circuit Diagram 12
Figure 15:Memory Stage Circuit Diagram 12
Figure 16:WB Stage Circuit Diagram 13
Figure 17:Main Control Unit Circuit ..... **Error! Bookmark not defined.**
Figure 18:Main Control Unit Code 17
Figure 19:PC Control Unit Circuit 18
Figure 20:PC Control Unit Code 18
Figure 21:Stall Circuit 19
Figure 22:Kill Circuit 19
Figure 23:Stall and Kill Unit Codes 20
Figure 24:Forwarding Circuit 21
Figure 25:Forwarding Code 21
Figure 26:Actual Simulation Test 1 Values 23
Figure 27: Simulation Test 1 WaveForm 23
Figure 28:Actual Simulation Test 2 Values 25
Figure 29:Stall Detected 25
Figure 30:Waveform For Simulation Test 2 26
Figure 31:Actual Simulation Test 3 Values 27
Figure 32:Waveform For Simulation Test 3 28
Figure 33: Team Work Distribution 29

Table Of Tables

Table 1:32-bit instruction format 1

Table 2:Control Signals Truth Table 15

Table 3: Control Signal Expression Table 16

Table 4:Simulation Test 1 Instructions Table 22

Table 5:Expected Result For Simulation Test 1 22

Introduction to Design and Implementation

Development Environment:

VS Code was used for writing and managing the code and everything else.

- **Simulation & Synthesis: Icarus Verilog enabled efficient testing and integration of processor modules.**
- **Waveform Analysis: GTKWave provided a detailed visual tool for signal timing and pipeline behavior verification.**
- **Modular Design: The processor was developed using Logisim in manageable circuits and components to aid in visualization and implementation.**
- **Hazard & Control Flow Management: The pipeline's behavior was verified to ensure proper handling of hazards and control instructions.**
- **Streamlined Development: The combination of these tools ensured an efficient, collaborative and correctly functioning pipelined processor design.**

Design's 32-bit format

• 6-bit opcode

• 4-bit Rd: Destination register

• 4-bit Rs: First source register

• 4-bit Rt: Second source register

• 14-bit Immediate: The immediate value, which is zero-extended for logical instructions and sign-extended otherwise

• For BZ, BLZ, BGZ, J, and CLL instructions, the target address is calculated by adding the current value of the PC to the sign extended immediate value.

Table 1:32-bit instruction format

Opcode^{6}	Rd^{4}	Rs^{4}	Rt^{4}	Imm^{14}

RTL Design and Instruction Encoding

Arithmetic and Logical Register-Based Instructions

Instruction	Meaning	Opcode Value
OR Rd, Rs, Rt	Reg(Rd) = Reg(Rs) | Reg(Rt)	0
ADD Rd, Rs, Rt	Reg(Rd) = Reg(Rs) + Reg(Rt)	1
SUB Rd, Rs, Rt	Reg(Rd) = Reg(Rs) - Reg(Rt)	2
CMP Rd, Rs, Rt	Reg(Rd) = 0 if(Reg(Rs) == Reg(Rt)) Reg(Rd) = -1 if(Reg(Rs) < Reg(Rt)) Reg(Rd) = 1 if(Reg(Rs) > Reg(Rt))	3

Figure 1: OR, ADD,SUB, CMP description and Opcode

1. **Fetch Stage**:
   instruction ←MEMORY[PC], PC ←PC + 1
2. **Decode Stage**:
   Data1 ←register[Rs], Data2 ←register[Rt]
3. **Execution Stage**:
   Result ←ALU(Data1, Data2)
4. **Memory Stage**:
   No memory access.
5. **Write-Back Stage**:
   register[Rd] ←Result

Arithmetic Immediate Instructions

ORI Rd, Rs, Imm	Reg(Rd) = Reg(Rs) | Imm	4
ADDI Rd, Rs, Imm	Reg(Rd) = Reg(Rs) + Imm	5

Figure 2:ORI,ADDI description and Opcode

1. **Fetch Stage**:
   instruction ←InstructionMemory [PC], PC ←PC + 1
2. **Decode Stage**:
   Data1 ←register[Rs], Data2 ←Sign/Zero-extend(Imm)
3. **Execution Stage**:
   Result ←ALU(Data1, Data2)
4. **Memory Stage**:
   No memory access.
5. **Write-Back Stage**:
   register [Rd] ←Result

Load Word Instruction (LW)

LW Rd, Imm(Rs)	Reg(Rd) = Mem(Reg(Rs) + Imm)	6

Figure 3: LW description and Opcode

1. **Fetch Stage**: instruction ← MEMORY[PC], PC ←PC + 1
2. **Decode Stage**: Base ← register[Rs], Offset ← Sign-extend(Imm)
3. **Execution Stage**: Address ← Base + Offset
4. **Memory Stage**: Data ← InstructionMemory [Address]
5. **Write-Back Stage**: register[Rd] ←Data

Store Word Instruction (SW)

SW Rd, Imm(Rs)	Mem(Reg(Rs) + Imm) = Reg(Rd)	7

Figure 4: SW description and Opcode

1. **Fetch Stage**:
   instruction ← InstructionMemory [PC], PC ←PC + 1
2. **Decode Stage**:
   Base ← register[Rs], Offset ← Sign-extend(Imm)
3. **Execution Stage**:
   Address ← Base + Offset
4. **Memory Stage**:
   InstructionMemory [Address] ← register[Rd]
5. **Write-Back Stage**:
   No register write-back.

Load Double Word Instruction (LDW)

LDW Rd, Imm(Rs)	In the first clock cycle: • Reg(Rd) = Mem(Reg(Rs) + Imm) In the second clock cycle: • Reg(Rd + 1) = Mem(Reg(Rs) + Imm + 1) • LDW (load double word) loads two consecutive words from memory to two consecutive destination registers in two consecutive clock cycles. • The destination register number in the instruction must be an even number. • If the processor detects an LDW instruction with an odd destination register number, it will throw an exception • When the processor detects an LDW instruction in the decode unit, it will not bring a new instruction in the next clock cycle, because in the next cycle, the processor will handle the second word	8

Figure 5: LDW description and Opcode

Cycle 1:

1. **Fetch Stage**:
   instruction ← InstructionMemory [PC], PC ← PC
2. **Decode Stage**:
   Base ← register[Rs], Offset ← Sign-extend(Imm)
3. **Execution Stage**:
   Address ← Base + Offset
4. **Memory Stage**:
   Data1 ← InstructionMemory [Address]
5. **Write-Back Stage**:
   register[Rd] ← Data1

Cycle 2:

1. **Fetch Stage**:
   instruction ← InstructionMemory [PC], PC ← PC + 1
2. **Decode Stage**:
   Base ← register[Rs], Offset ← Sign-extend(Imm)
3. **Execution Stage**:
   Address ← Base + Offset + 1
4. **Memory Stage**:
   Data2 ← InstructionMemory [Address]
5. **Write-Back Stage**:
   register[Rd+1] ← Data2

Store Double Word Instruction (SDW)

SDW Rd, Imm(Rs)	In the first clock cycle: • Reg(Rd) ➔ Mem(Reg(Rs) + Imm) In the second clock cycle: • Reg(Rd + 1) ➔ Mem(Reg(Rs) + Imm + 1) • SDW (store double word) stores the content of two consecutive source registers into two consecutive words in memory in two consecutive clock cycles. • The source register number in the instruction must be an even number. • If the processor detects an SDW instruction with an odd source register number, it will throw an exception • When the processor detects an SDW instruction in the decode unit, it will not bring a new instruction in the next cycle, because in the next cycle, the processor will handle the second word	9

Figure 6: SDW description and Opcode

Cycle 1:

1. **Fetch Stage**:
   instruction ←InstructionMemory[PC], PC ←PC
2. **Decode Stage**:
   Base ←register[Rs], Offset ←Sign-extend(Imm)
3. **Execution Stage**:
   Address ←Base + Offset
4. **Memory Stage**:
   InstructionMemory [Address] ←register[Rd]

5. **Write-Back Stage**:
   No register write-back.

Cycle 2:

1. **Fetch Stage**:
   instruction ← InstructionMemory [PC], PC ← PC + 1
2. **Decode Stage**:
   Base ←register[Rs], Offset ←Sign-extend(Imm)
3. **Execution Stage**:
   Address ← Base + Offset + 1
4. **Memory Stage**:
   Memory[Address] ← register[Rd+1]
5. **Write-Back Stage**:
   No register write-back.

Branch Instructions *(BZ, BGZ, BLZ)*

BZ Rs, Label	if (Reg(Rs) == 0) Next PC = branch target else Next PC = PC + 1	10
BGZ Rs, Label	if (Reg(Rs) > 0) Next PC = branch target else Next PC = PC + 1	11
BLZ Rs, Label	if (Reg(Rs) < 0) Next PC = branch target else Next PC = PC + 1	12

Figure 7: BZ, BGZ, BLZ description and Opcode

1. **Fetch Stage**:
   instruction ← MEMORY[PC], PC ← PC + 1
2. **Decode Stage**:
   Data1 ← register[Rs], Data2←0
   - If condition on Rs (== 0, > 0, < 0) is true, then:
     - PC ← PC + Sign-extend(Imm)
     - Flush the next instruction.
   - Else: PC ← PC + 1
3. **Execution Stage**:
   No ALU operation.

4. **Memory Stage**:
   No memory access.
5. **Write-Back Stage**:
   No register write-back.

Jump Register Instruction (JR)

JR Rs	Jump to the target address stored in the register Rs	13

Figure 8: JR Rs description and Opcode

1. **Fetch Stage**:
   instruction ← MEMORY[PC], PC ← PC + 1
2. **Decode Stage**:
   PC ← register[Rs]
   - Kill the fetched instruction.
3. **Execution Stage**:
   No ALU operation.
4. **Memory Stage**:
   No memory access.
5. **Write-Back Stage**:
   No register write-back.

Jump Instruction (J)

J Label	Unconditionally jump to the target address specified by the label	14

Figure 9: J Label description and Opcode

1. **Fetch Stage**:
   instruction ← MEMORY[PC], PC ← PC + 1
2. **Decode Stage**:
   PC ← PC + Sign-extend(Imm)
   - Kill the fetched instruction.
3. **Execution Stage**:
   No ALU operation.
4. **Memory Stage**:
   No memory access.

5. **Write-Back Stage**:
   No register write-back.

Call Instruction (CLL)

CLL Label	Jump to the function at the specified label and store the return address in register R14	15

Figure 10: CLL Label description and Opcode

1. **Fetch Stage**:
   instruction ← MEMORY [PC], PC ← PC + 1
2. **Decode Stage**:
   - PC ← PC + Sign-extend(Imm)
   - Save return address: register[R14] ← PC + 1
   - Kill the fetched instruction.
3. **Execution Stage**:
   No ALU operation.
4. **Memory Stage**:
   No memory access.
5. **Write-Back Stage**:
   register[R14] ← return address

Datapath

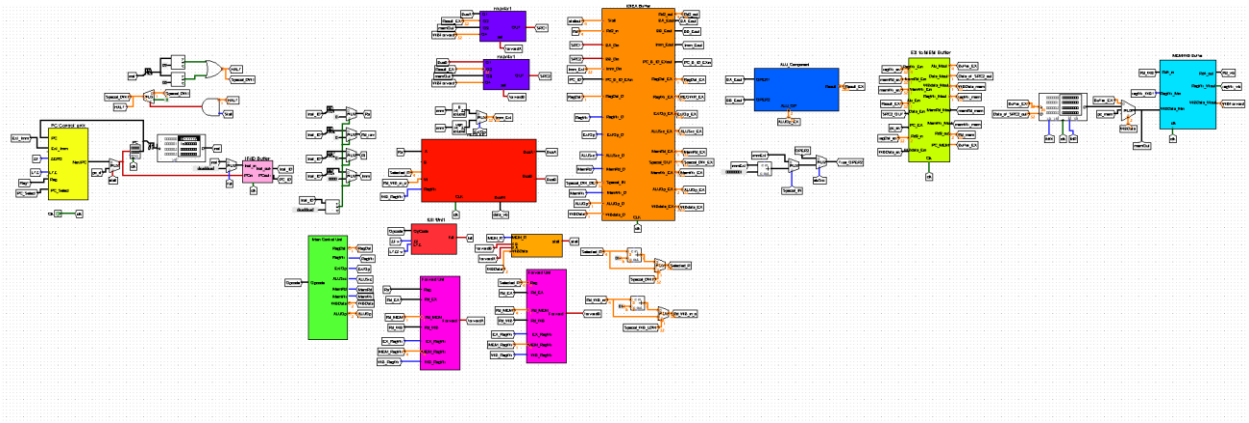


Figure 11: Main Datapath Circuit Diagram

The above figure shows the complete Datapath of our Pipelined Processor:

1. Fetch Stage

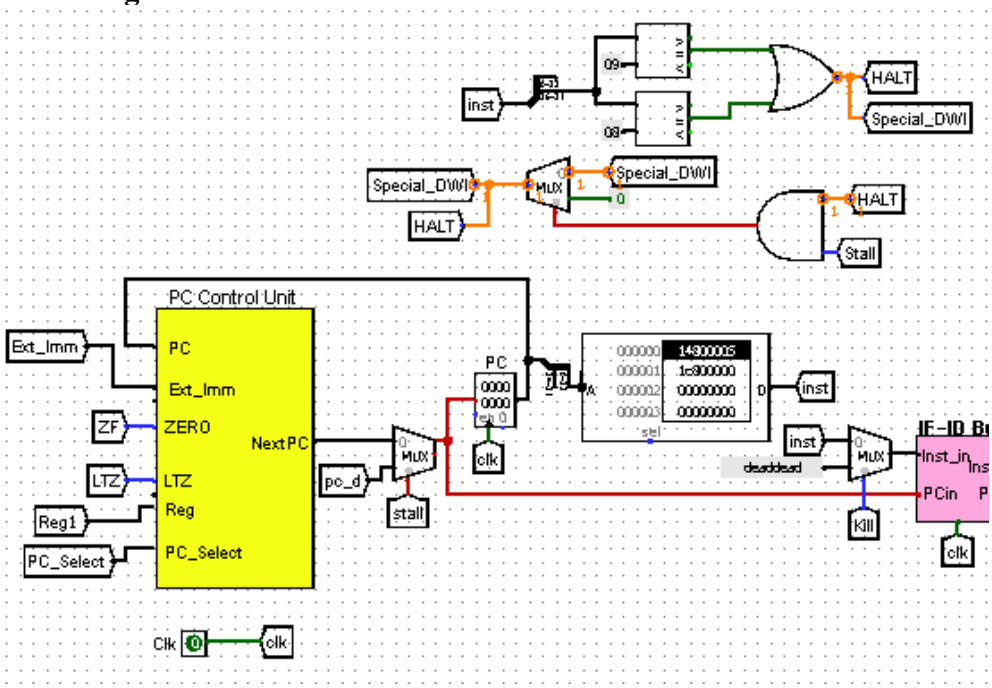


Figure 12:Fetch Stage Circuit Diagram

This is the initial stage of the pipeline, where the instruction is fetched from the instruction memory using the current value of the Program Counter (PC). The PC Control Unit determines the next PC value with a default of PC +1. However, for control-flow instructions such as branches, jumps, calls, and returns, the PC is updated differently based on the instruction type.

2. Decode Stage

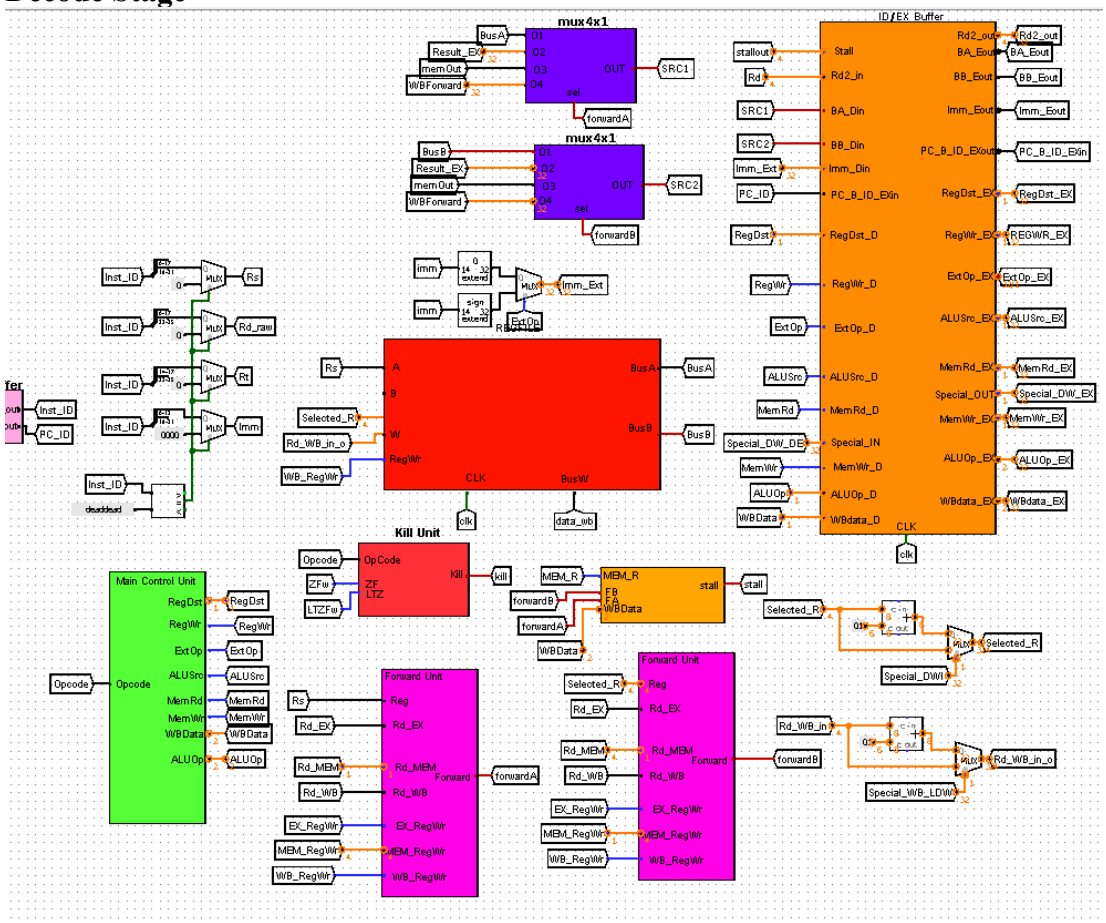


Figure 13:Decode Stage Circuit Diagram

Once fetched, the instruction moves to the decode stage, where it is broken down into its components to determine the operation type. The opcode is extracted and sent to the control unit to generate the necessary control signals. During this stage, the required source registers are read from the register file. Additionally, this stage is responsible for handling data hazards, stalls, kills, and data forwarding to maintain pipeline correctness.

3. Execute Stage

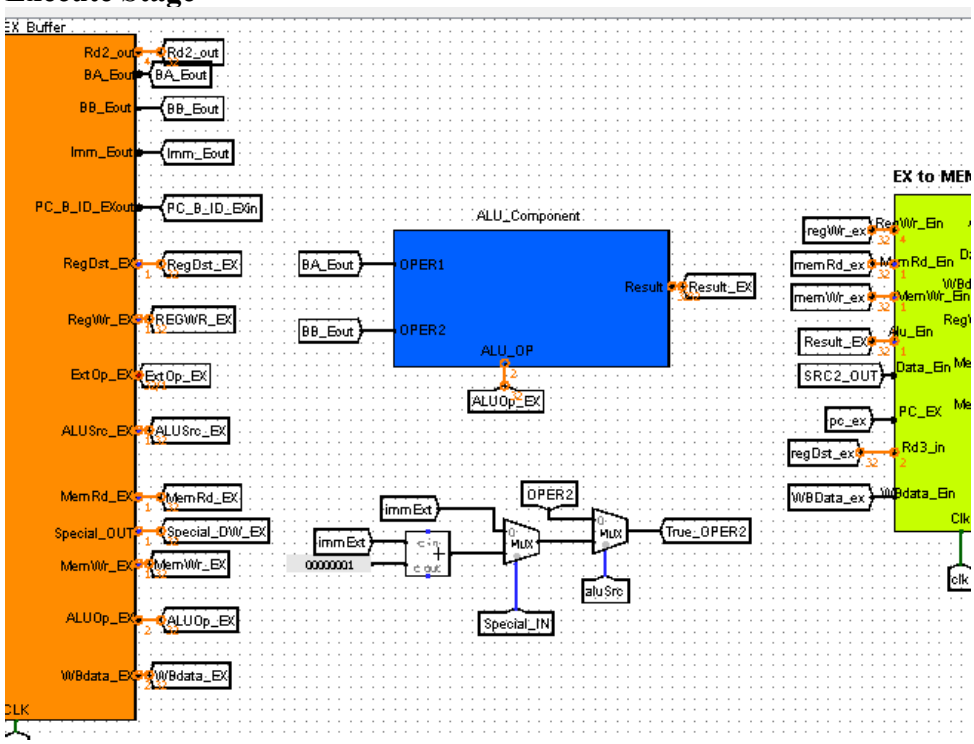
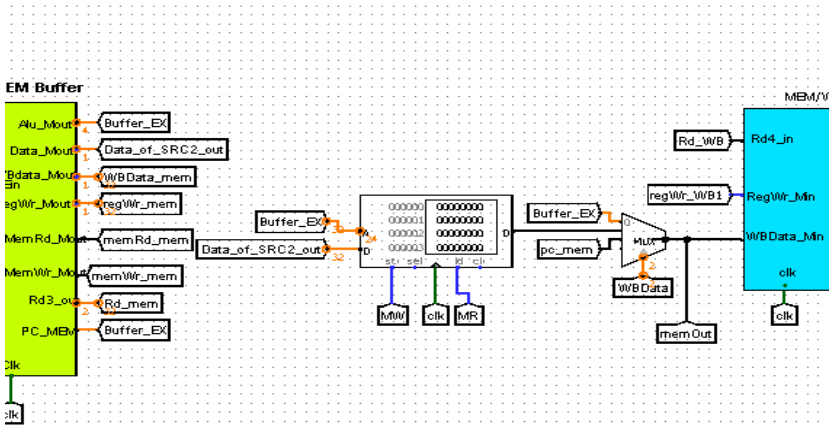


Figure 14:Execute Stage Circuit Diagram

In this stage, the processor performs any required computations. For most instructions, this involves using the ALU to perform arithmetic or logical operations, or to calculate memory addresses. If the instruction does not require an ALU operation, its data is passed forward to the memory stage in the next clock cycle.

4. Memory Stage



The memory stage handles load and store instructions. Load instructions read data from instruction memory, while store instructions write data to instruction memory. Even though only memory-related instructions actively use this stage, all instructions pass through it for proper pipeline alignment.

5. Write-Back Stage

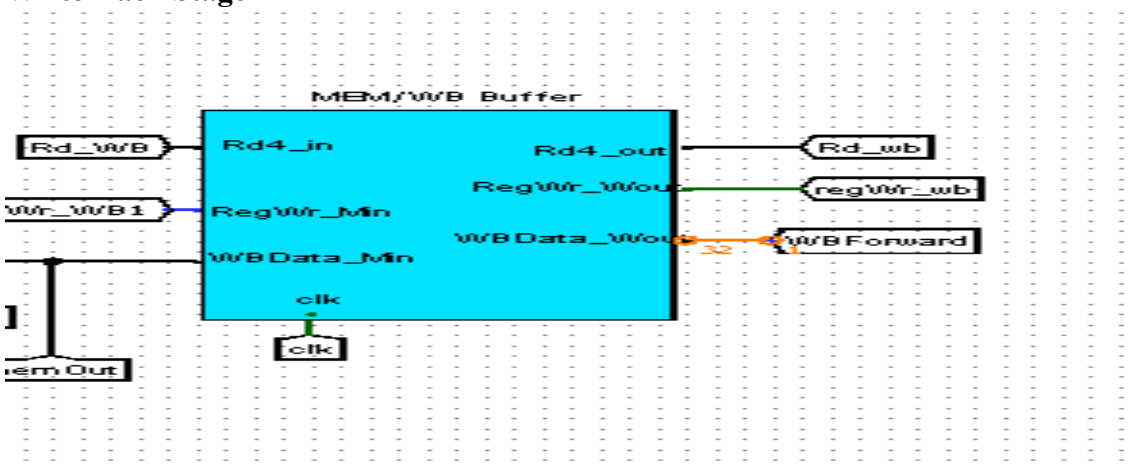


Figure 16:WB Stage Circuit Diagram

This is the final pipeline stage. If the instruction is designed to write to a register and the write-enable control signal is asserted, the result from the execution or memory stage is written back to the appropriate destination by propagating the signals back to Register-File in decode.

Datapath Components Overview

Instruction Memory:
Using the current value of the Program Counter (PC), the instruction memory fetches the corresponding instruction from memory.

Program Counter (PC) Control:
The PC Control Unit calculates the address of the next instruction. Normally, it increments the PC by one (PC + 1), but for control instructions like Jump, Branches, CLL, and it computes a new target address using the sign-extended immediate value and for JR, the next PC is what's stored in the register.

Pipeline Buffers:
Located between pipeline stages, these buffers capture inputs from one stage and deliver outputs to the next at each rising clock edge, maintaining data and Control signal flow across the pipeline stages.

Control Unit:
Using a code that works like a splitter (no designated module), it Breaks down the fetched instruction into components—opcode, source registers (Rs, Rt), destination register (Rd), and immediate value (Imm). The control unit then generates the appropriate control signals based on the opcode.

Register File:
This unit provides the values stored in the Rs and Rt registers. If writing back is enabled (via the RegWr signal), it writes to destination register (Rd) during the Write Back stage.

Forwarding, Stalling, and Killing Mechanisms:
To address data and control hazards, the forwarding unit provides necessary data early, preventing stalls. The stall unit pauses pipeline execution when required, while the kill unit removes incorrect instructions from the pipeline.

Arithmetic Logic Unit (ALU):
The ALU performs calculations based on the AluOp control signal. It handles operations like bitwise OR, addition, subtraction and comparisons.

Data Memory:
This module is responsible for memory access. For load instructions like LW and LDW, it reads data from Data Memory and forwards it to the MEM/WB buffer. For store instructions such as SW and SDW, it writes data from registers into Data memory.

Write Back (WB) Stage:
This final pipeline stage propagates the signals to the other stages to allow the pipeline to function correctly such as writing into the register file. The data may originate from the ALU, memory, or the incremented PC, based on the WBData control signal.

Control Unit Signals

Control Signals Truth table

Chosen some Signals as 1 or 0 even if it can work as X for some instructions.

Instruction	RegDst	RegWr	ExtOp	ALUSrc	MemRd	MemWr	WBdata		ALUOp		Hex value
OR	0	1	1	0	0	0	0	0	0	0	180
ADD	0	1	1	0	0	0	0	0	0	1	181
SUB	0	1	1	0	0	0	0	0	1	0	182
CMP	0	1	1	0	0	0	0	0	1	1	183
ORI	0	1	1	1	0	0	0	0	0	0	1C0
ADDI	0	1	1	1	0	0	0	0	0	1	1C1
LW	0	1	1	1	1	0	0	1	0	1	3E5
SW	1	0	1	1	0	1	0	0	0	1	2D1
LDW	0	1	1	1	1	0	0	0	0	1	3E1
SDW	1	0	1	1	0	1	0	0	0	1	2D1
BZ	0	0	1	0	0	0	0	0	0	0	80
BGZ	0	0	1	0	0	0	0	0	0	0	80
BLZ	0	0	1	0	0	0	0	0	0	0	80
JR	0	0	0	1	0	0	0	0	0	0	C0
J	0	0	1	0	0	0	0	0	0	0	0
CLL	0	1	1	0	0	0	1	0	0	0	188

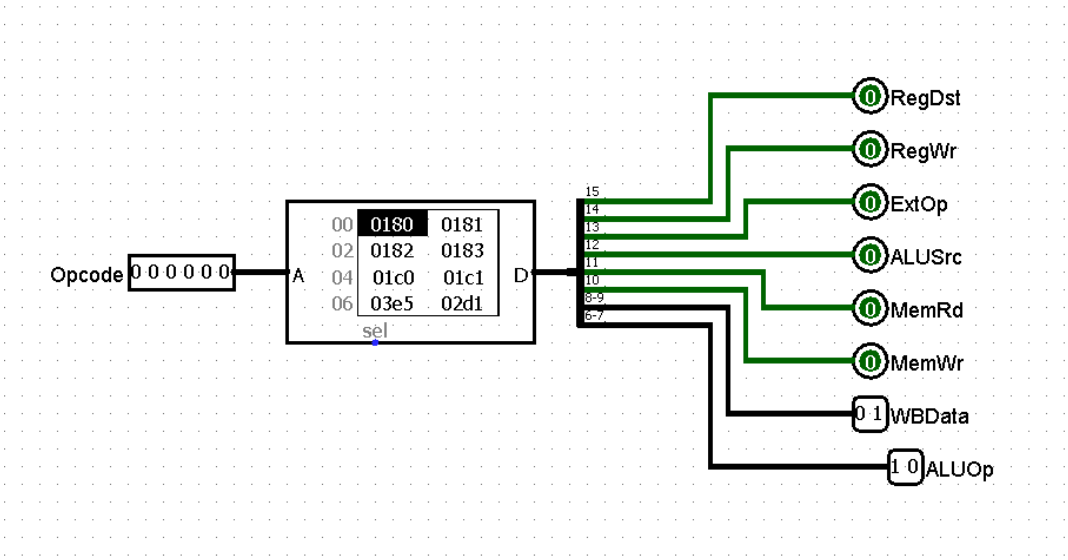
Table 2:Control Signals Truth Table

Control Signal	Boolean Expression
RegDst	SW | | SDW
RegWr	OR | | ADD | | SUB | | CMP | | ORI | | ADDI | | LW | | LDW | | CLL
Ext_Op	ADDI | | LW | | SW | | LDW | | SDW | | BZ | | BGZ | | BLZ | | J | | CLL
ALUSrc	ORI | | ADDI | | LW | | SW | | LDW | | SDW
MemRd	LW | | LDW

MemWr	SW | | SDW		
WBdata	**Value**	**Operation**	**Instructions**
	00	Save ALU result to register	Arithmetic Instructions
	01	Get memory result to register	LDW, LW
	10	Save PC to register 14	CLL
ALUOp	**Value**	**Operation**	**Instructions**
	2'b00	Bitwise OR	OR, ORI
	2'b01	Addition	ADD, ADDI, LW, SW, LDW, SDW
	2'b10	Subtraction	SUB
	2'b11	Comparison	CMP

Table 3: Control Signal Expression Table

Main Control Unit




```
module Control_Unit(input [5:0] Opcode,output reg RegDst, RegWr, ExtOp, ALUSrc, MemRd, MemWr,output reg [1:0] WBdata, ALUOp);


  reg [9:0] CU_Memory [15:0];

  initial begin
    // READ FROM Control Signals File
    $readmemb("Control_Signals.dat",CU_Memory);
  end

  always @(Opcode)begin
    // Output the Control Signal according to OPCODE to each reg
    if (Opcode < 16)begin
      RegDst  = CU_Memory[Opcode][9];
      RegWr   = CU_Memory[Opcode][8];
      ExtOp   = CU_Memory[Opcode][7];
      ALUSrc  = CU_Memory[Opcode][6];
      MemRd   = CU_Memory[Opcode][5];
      MemWr   = CU_Memory[Opcode][4];
      WBdata  = CU_Memory[Opcode][3:2];
      ALUOp   = CU_Memory[Opcode][1:0];
    end
    else
      {RegDst, RegWr, ExtOp, ALUSrc, MemRd, MemWr, WBdata, ALUOp} = 0;

  end


endmodule
```

The main control unit takes a 6-bit opcode and uses it to look up control signals from a predefined Data file (Control_Signals.dat). These signals →RegWr, ALUSrc, ExtOp, MemRd, MemWr, WBdata, ExtOp, and ALUOp—are used throughout the Datapath stages.

PC Control Unit

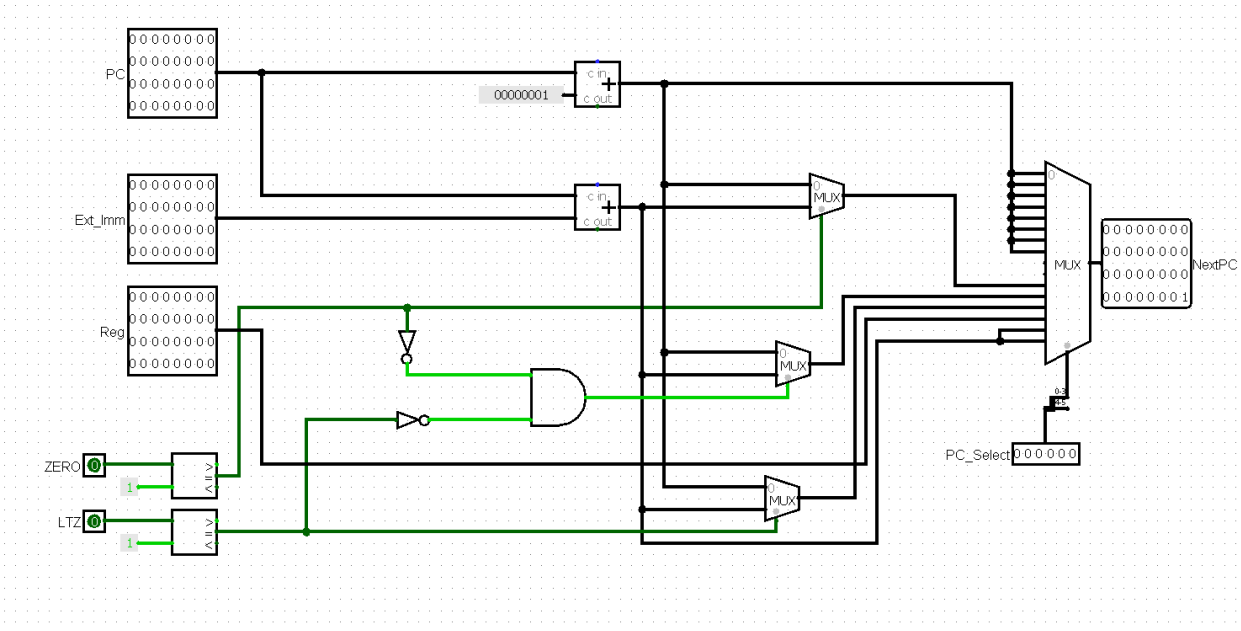


Figure 19:PC Control Unit Circuit

```
module PC_Control_Unit(
    input  [31:0] currentPC,
    input  [31:0] Reg,
    input  signed [31:0] Ext_Imm,
    input         ZERO,
    input         LTZ,
    input  [5:0]  PC_Select,
    output reg [31:0] NextPC,
    input Special_DW
);
    // THIS UNIT CONTROLS THE PROGRAM COUNTER
    // initial $monitor("2.PC_Control_Unit: PC_Select=%d, currentPC=%0d, Reg=%0d, Ext_Imm=%0d, Z=%b, LTZ=%b => NextPC=%0d",
    //  PC_Select, currentPC, Reg, Ext_Imm, ZERO, LTZ, NextPC);
    initial NextPC = 1'b0;
    always @(*) begin

        case (PC_Select)
                `OR, `ADD, `SUB, `CMP, `ORI, `ADDI, `LW, `SW: // THESE INSTRUCTIONS DO NOT CHANGE THE ORDER OF EXECUTION SO PC+1
                NextPC = currentPC + 32'd1;
            `J,`CLL: NextPC = currentPC + Ext_Imm-1;  // NEXT PC IS DETERMINED BY THE GIVEN IMMIEDIATE AND CURRENT PC
            `JR: NextPC = Reg; // NEXT PC IS STORED IN REG 14
            `BZ: begin  // CONDINTIONAL BRANCHES BGZ BLZ BZ
                if (ZERO) begin
               NextPC = currentPC + Ext_Imm-1;
               $display("BRANCH TAKEN: NextPC = %0d (currentPC = %0d + Ext_Imm = %0d)",
                        NextPC, currentPC, Ext_Imm-1);
            end
                else
                    NextPC = currentPC + 32'd1;
                end
            `BLZ: begin
                if (LTZ)
                    NextPC = currentPC + Ext_Imm-1;
                else
                    NextPC = currentPC + 32'd1;
                end
            `BGZ: begin
                if (ZERO == 0 && LTZ == 0) begin
                    NextPC = currentPC + Ext_Imm-1;
                    $display("BRANCH TAKEN: NextPC = %0d (currentPC = %0d + Ext_Imm = %0d)",
                      currentPC + Ext_Imm, currentPC, Ext_Imm);
                end
                  else
                    NextPC = currentPC + 32'd1;
                end
            default: NextPC = currentPC+1;
        endcase
    end

endmodule
```

Figure 20:PC Control Unit Code

The PC Control Unit decides the next value of the program counter based on the opcode and condition flags. Normally, for non-control instructions, it simply moves to the next instruction by adding 1 to the current PC. However, for branch instructions like BZ, BGZ, and BLZ, it adds an offset (Ext_Imm - 1) to the PC only if the branch condition is met. The JR instruction changes the PC to the value stored in register Reg. For jump (J) and call (CLL) instructions, the PC is updated using the immediate value.

Stall :

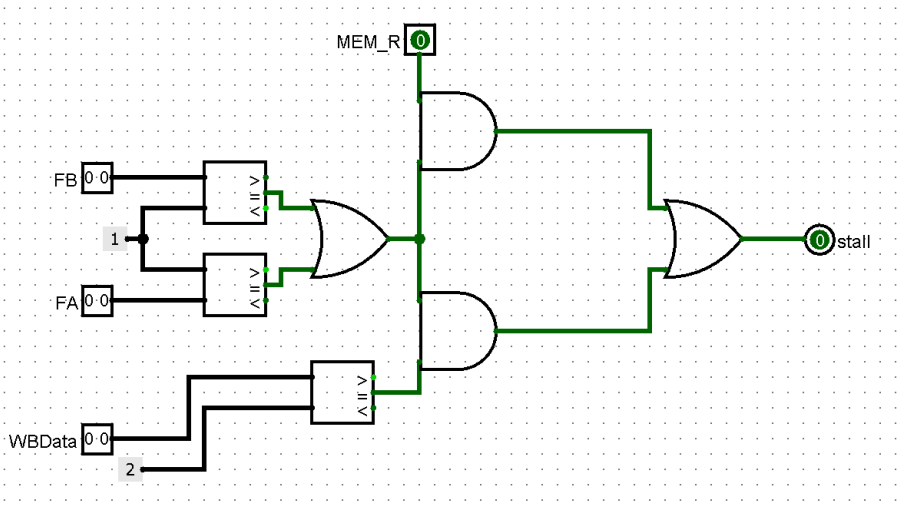


Figure 21:Stall Circuit

Kill:

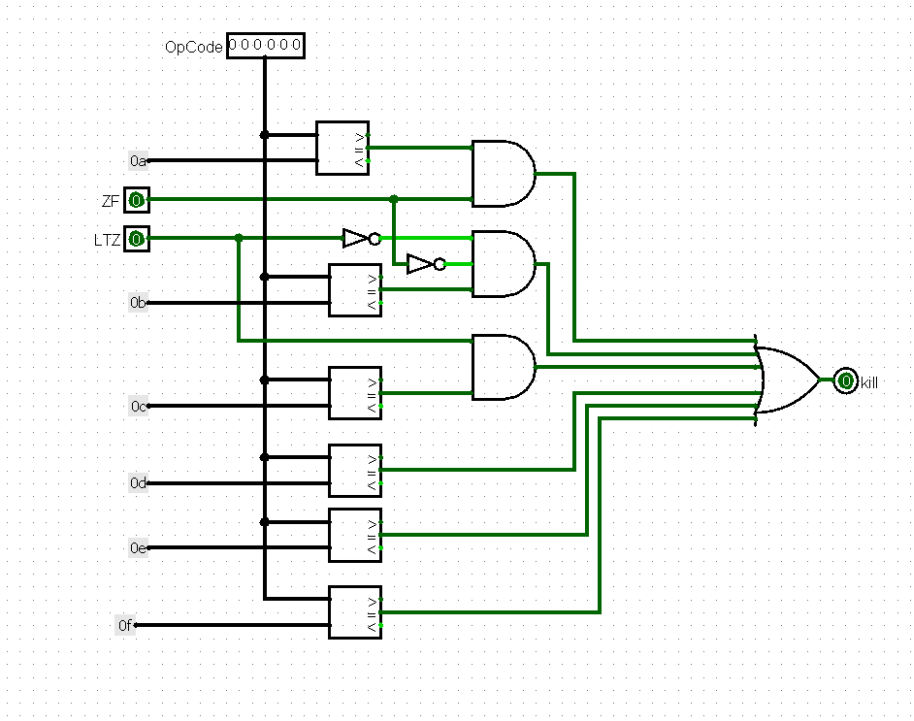


Figure 22:Kill Circuit

```
module Stall(input MEM_R,input [1:0] FA,FB,WBData,output reg stall);
initial stall = 0;

always @(*) begin

  if ((FA == 1 || FB == 1) && MEM_R) // CHECK IF THERE IS A CASE OF FORWARDING WHILE THE MEM_R IS FORWARDED FROM EXECUTION STAGE
    stall = 1;
  else if((FA == 1 || FB == 1) && WBData==2'b10) // STALL IF THERE IS FORWARDING AND CLL INSTRUCTION
    stall = 1;
  else
    stall=0;

  if (stall)begin
      $display("Stall Check at time %0t: FA=%b, FB=%b, MEM_R=%b, WBData=%b => stall=%b",
              $time, FA, FB, MEM_R, WBData, stall);
  end
end
endmodule


    // if (MEM_R !==1'bx)
    //       $display("STALL: FA = %b, FB = %b, MEM_R = %b => stall = %b", FA, FB, MEM_R, stall);
//       if (MEM_R !==1'bx)
//   $display("NO STALL: FA = %b, FB = %b, MEM_R = %b => stall = %b", FA, FB, MEM_R, stall);


module Kill(input [5:0] OpCode,input ZF,LTZ,output reg kill);
    initial kill = 0;
    always @(*) begin
    if ((OpCode == `BZ) && ZF==1) kill = 1; // KILL CYCLE IF A BRANCH IS TAKEN // OR UNCONDINATIONAL BRANCH (JUMP)
    else if ((OpCode == `BGZ) && (LTZ != 1 && ZF != 1)) kill = 1;
    else if ((OpCode == `BLZ) && LTZ == 1) kill = 1;
    else if (OpCode == `J || OpCode == `JR || OpCode == `CLL) kill = 1;
    else kill = 0;
    // $display("Kill Module Debug -- Time: %0t | OpCode: 0x%02h | ZF: %b | LTZ: %b | kill: %b",
    //        $time, OpCode, ZF, LTZ, kill);
  end

endmodule
```

Figure 23:Stall and Kill Unit Codes

The stall and kill units work together to manage pipeline hazards and ensure correct program execution. The stall unit focuses on detecting data hazards, such as load-use cases and special write-back situations like with the CLL instruction. It monitors whether the instruction in the Execute stage is reading from memory (MEM_R), and checks if the next instruction depends on that result through its source operands through forwarding (FA or FB). If a dependency is found, the stall signal is triggered to pause the pipeline for one cycle, avoiding incorrect data usage.

Meanwhile, the kill unit deals with control hazards caused by branches and jumps. It evaluates the opcode and condition flags (ZF and LTZ) of the current instruction to decide if the instruction in the fetch or decode stage should be invalidated. If a control transfer like J, or a taken branch (BZ, BGZ, BLZ) is identified, the kill signal is asserted to flush the incorrect instruction and keep the program flow accurate.

Forwarding Unit

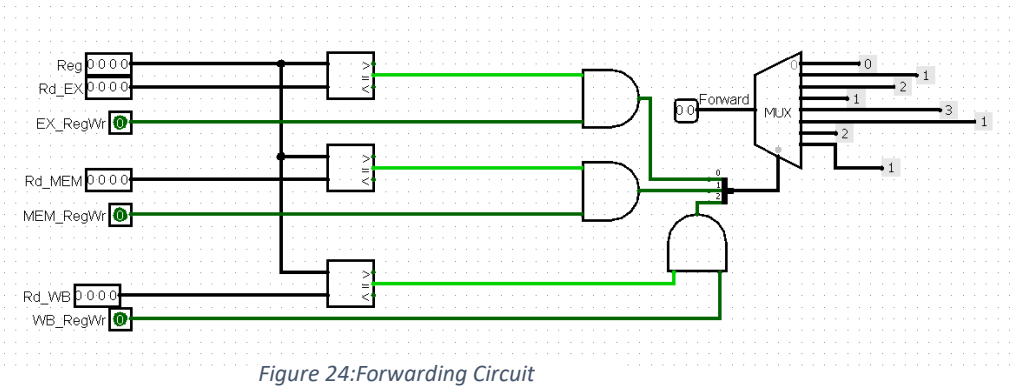


Figure 24:Forwarding Circuit

```
module Forward(
  input  [3:0] Reg, Rd_EX, Rd_MEM, Rd_WB,
  input        EX_RegWr, MEM_RegWr, WB_RegWr,
  output reg [1:0] Forward
);
  always @(*) begin

  if ((Reg == Rd_MEM) && MEM_RegWr) begin // if Reg in MEM Stage and theres a write operation FORWARD
    Forward = 2'b10;
    // $display("Forward from MEM: R%d", Reg);
  end
  else if ((Reg == Rd_EX) && EX_RegWr) begin // if Reg in EX Stage and theres a write operation FORWARD
    Forward = 2'b01;
    // $display("Forward from EX: R%d", Reg);
  end
  else if ((Reg == Rd_WB) && WB_RegWr) begin // if Reg in WB Stage and theres a write operation FORWARD
    Forward = 2'b11;
    // $display("Forward from WB: R%d", Reg);
  end
  else begin // NO FORWARDING
    Forward = 2'b00;
  end
   if (Forward != 2'b00) begin
    // $display("Forward Check: Reg=%d, Rd_EX=%d, Rd_MEM=%d, Rd_WB=%d, EX_RegWr=%b, MEM_RegWr=%b, WB_RegWr=%b => Forward=%b",
    //     Reg, Rd_EX, Rd_MEM, Rd_WB, EX_RegWr, MEM_RegWr, WB_RegWr, Forward);
 end
end

endmodule
```

Figure 25:Forwarding Code

This Verilog module defines a forwarding unit that helps resolve data hazards by selecting the latest valid value for a source register (**Reg**). It compares this source register with the destination registers from the Execute (**Rd_EX**), Memory (**Rd_MEM**), and Write-Back (**Rd_WB**) stages, and checks if each stage is enabled to write using the signals **EX_RegWr**, **MEM_RegWr**, and **WB_RegWr**. Based on these checks, the module assigns a value to the Forward signal: '01' for the Execute stage, '10' for the Memory stage, '11' for the Write-Back stage, and '00' when no forwarding is required. This ensures that instructions use the most recent data available, keeping the pipeline running without delays

Processor Simulation and Verification

Simulation Test 1: Integer Divider using given instructions

The Objective is to compute the **quotient** and **remainder** of two numbers by **repeatedly subtracting** the **divisor** from the **dividend** and incrementing a counter for how many times the subtraction is possible (which becomes the result of the division).

Instruction	Hex	Description
ADDI R0, R0, 0	14000000	Initialize R0 to 0 to store **quotient**. (NOT Required as all Registers start with 0)
ADDI R1, R0, 17	14400011	Set R1 = 17 (**dividend**.)
ADDI R2, R0, 5	14800005	Set R2 = 5 (**divisor**.)
ADD R3, R1, R0	04C40000	Copy R1 into R3 to use as a **temp for dividend.**
CLL 4	3C000004	Save current address at R14 and Jump to Pc+4
SW R0, 0(R14)	1C380000	Store **quotient** (initially 0) to memory [0 + R14].
SW R3, 1(R14)	1CF80001	Store current remainder to memory [1 + R14].
J 10	3800000A	Jump to instruction at address 10 **ending the Program.**
SUB R4, R3, R2	090C8000	Subtract divisor (R2) from remainder (R3), store in R4.
BLZ R4, 4	30100004	If the result in R4 is less than zero, break loop (JUMP to end of Loop).
ADDI R3, R4, 0	14D00000	Update remainder R3 = R4.
ADDI R0, R0, 1	14000001	Increment quotient (R0).
J -4	38003FFC	Jump back to the start of the loop (SUB instruction).
JR R14	34380000	Return to Address stored at R14.

Table 4:Simulation Test 1 Instructions Table

Expected Value

Location	Expected Value
R0	3
R1	17
R2	5
R3	2
R4	-3
R14 (RA)	5
Memory [5]	3
Memory [6]	2

Table 5:Expected Result for Simulation Test 1

Simulation Values:

```
[Time 285000] Register File:
  -----------------------------------------
  | R0 =   3 | R7  =   0 |
  | R1 =  17 | R8  =   0 |
  | R2 =   5 | R9  =   0 |
  | R3 =   2 | R10 =   0 |
  | R4 =  -3 | R11 =   0 |
  | R5 =   0 | R12 =   0 |
  | R6 =   0 | R13 =   0 |
  -----------------------------------------
  | R14 (RA) =    5      |
  -----------------------------------------
WROTE 3 TO MEM[5]
```

```
M[0] = 0
M[1] = 0
M[2] = 0
M[3] = 0
M[4] = 0
M[5] = 3
M[6] = 2
M[7] = 0
M[8] = 0
M[9] = 0
M[10] = x
```

Figure 26:Actual Simulation Test 1 Values

As shown in the figure above, all values align with the expected values.

Waveform Generation:

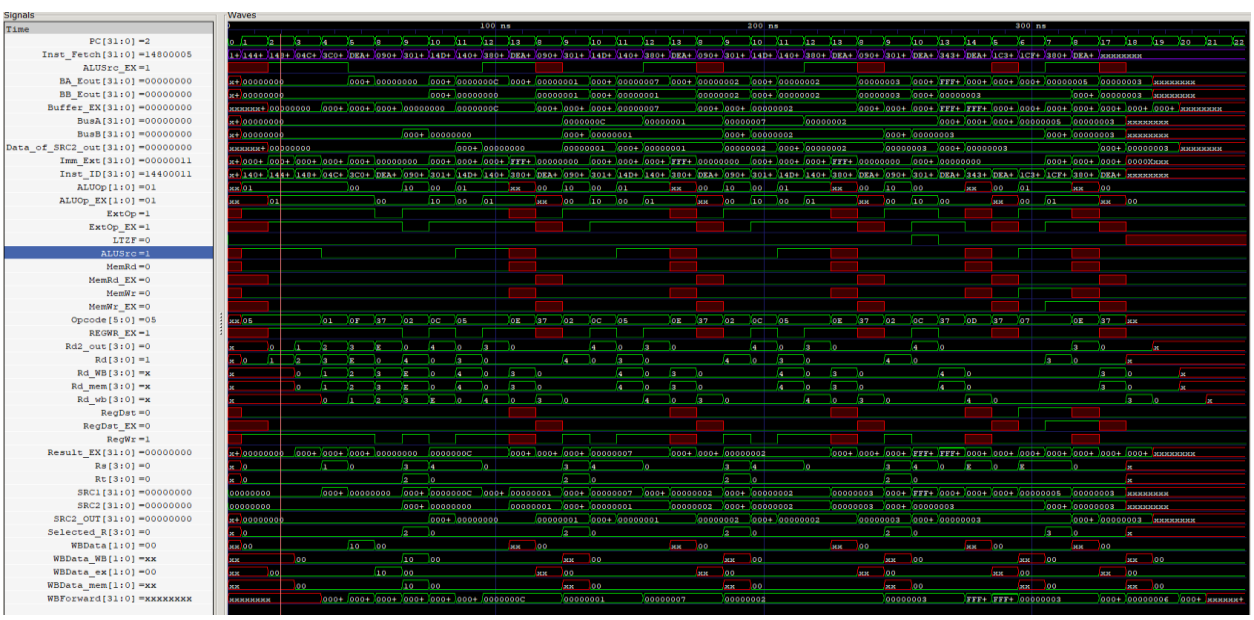


Figure 27: Simulation Test 1 Waveform

Simulation Test 2: General Instructions with Stall

This test represents only a sequence of general-purpose assembly instructions with a predetermined stall at BGZ designed to test an ability to identify and handle various types of stalls.

Instruction	Value	Description
ADDI R1 R0 10	1440000A	Add immediate value 10 to register R0, store in R1
ADDI R2 R0 20	14800014	Add immediate value 20 to register R0, store in R2
OR R3 R1 R2	00C48000	Perform bitwise OR on R1 and R2, store in R3
CMP R4 R3 R1	0D0C4000	Compare R3 and R1, store result in R4
BZ R4 2	28100002	Branch if zero based on R4, offset by 2 instructions
ORI R5 R5 100	11540064	Perform bitwise OR with immediate 100
SW R5 0(R0)	1D400000	Store word from R5 at memory [0 + R0].
LW R6 0(R0)	19800000	Load word from memory [0 + R0]. into R6
BGZ R6 4	2C180004	Branch if greater than zero based on R6, offset by 4 instructions
ADDI R7 R0 100	15C00064	Add immediate value 100 to register R0, store in R7

Table 6:Simulation Test 2 Instructions Table

Expected Value

Location	Expected Value
R0	0
R1	10
R2	20
R3	30
R4	1
R5	100
R6	100
R7	0
Memory [0]	100

R7 is 0 because of stall prevented last Instruction ADDI R7, R0, 100 15C00064

Table 7:Expected Result For Simulation Test 2

Simulation Values:

```
[Time 115000] Register File:                    WROTE 100 TO MEM[0]
  ----------------------------------------      M[0] = 100
  | R0 =   0 | R7  =   0 |                     M[1] = 0
  | R1 =  10 | R8  =   0 |                     M[2] = 0
  | R2 =  20 | R9  =   0 |                     M[3] = 0
  | R3 =  30 | R10 =   0 |                     M[4] = 0
  | R4 =   1 | R11 =   0 |                     M[5] = 0
  | R5 = 100 | R12 =   0 |                     M[6] = 0
  | R6 = 100 | R13 =   0 |                     M[7] = 0
  ----------------------------------------      M[8] = 0
  | R14 (RA) =    0      |                     M[9] = 0
  ----------------------------------------      M[10] = x
```

Figure 28:Actual Simulation Test 2 Values

Stall Detection:

```
================ Cycle 85000 =================
IF->| PC = 8          | Instruction = 2c180004 | Opcode 11 |S = 0 |K = 0
SPDW = 0
ID->| PC = 7          | ImmExt = 0          | Opcode = 06 | ZF= 1 LTZF = 0
    | Rs = 0 | Rt = 0 | Rd = 6 | Selected Reg = 0
    | src1 = 0          | src2 = 0          SPDW = 0
    | RegDst = 0 | ALUSrc = 1 | ALUOp = 01 | RegWr = 1 | MemRd = 1 | MemWr = 0 | WBData = 01
EX->| PC = 6          | ALUResult = 0          | DATA OF SRC2 = 100 , SPDW = 0
WARNING: design.v:140: missing argument for $display<%b>.
    | regDst = 5 | WBData = 00 | ZeroFlag = <%b>
    | RegWr = 0 | MemRd = 0 | MemWr = 1 | ALUOp = 01
MEM->| PC = 5          | MemOut = 100
     | regDst = 5 | MemRd = 0 | MemWr = 0 | RegWr = 1 | WBData = 00
WB-> | PC = 4          | Data = 1          | RegDst = 0 | RegWr = 0
PCs: IF=8 ID=7 EX=6 MEM=5 WB=4
Special_DW pipeline signals: Fetch=0, D=0, DE=0, EM=0, M=0, MW=0, WB=0
Time: 85000 | MemRd: D=1, ID_EX=0, EX=0, EX_MEM=0, MEM=0
Stall Check at time 85000: FA=01, FB=00, MEM_R=1, WBData=00 => stall=1
RegFile: Writing 100 to R5 at time 95000
WROTE 100 TO MEM[0]
M[0] = 100
M[1] = 0
M[2] = 0
M[3] = 0
M[4] = 0
M[5] = 0
M[6] = 0
M[7] = 0
M[8] = 0
M[9] = 0
M[10] = x
```

Figure 29:Stall Detected

As shown in the figure above, all Simulation values and conditions are met and align with the expected values.

Waveform Generation:

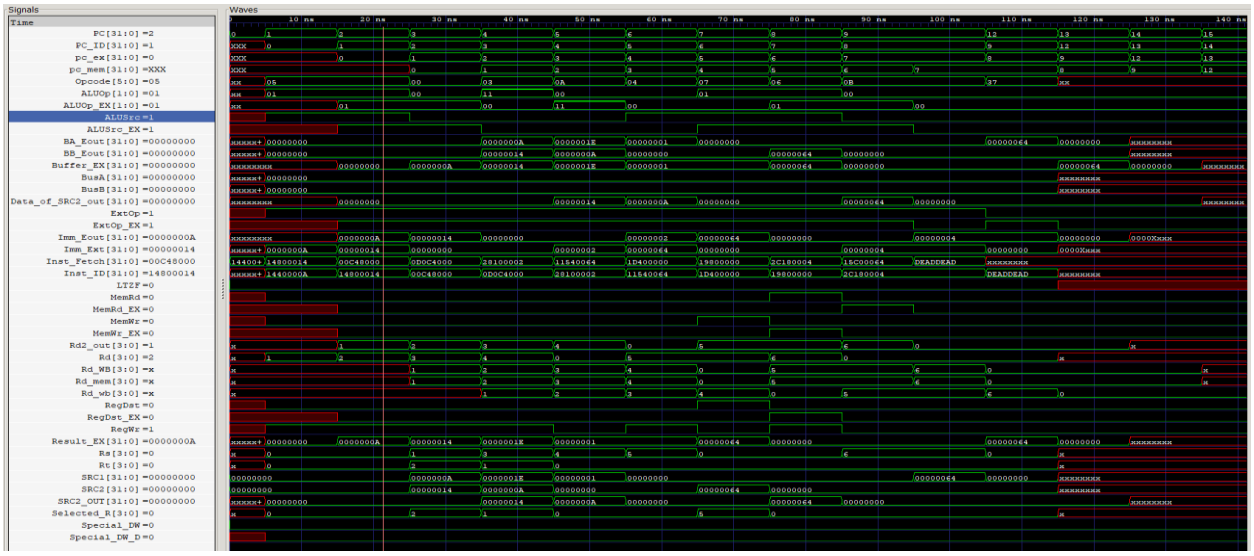


Figure 30:Waveform for Simulation Test 2

Simulation Test 3: General LDW SDW implementation

We will use our student numbers to test if we can load and store double consecutive memory or register addresses. (1220214, 1220050 (we will consider this as 122050 to simplify the testcase).

Instruction	Value	Description
ADDI R0 R0 1220	140004C4	Add immediate value 1220 to **R0.**
ADDI R1 R1 214	144400D6	Add immediate value 214 to **R1.**
ADDI R2 R2 1220	148804C4	Add immediate value 1220 to **R2**
ADDI R3 R3 50	14CC0032	Add immediate value 50 to **R3**
SDW R0 0(R10)	24240000	Store double word from **R0** to memory [0 + R10] and store **R1** to Memory [0 + R10 + 1]
SDW R2 2(R10)	24A40002	Store double word from **R2** to memory [2 + R10] and store **R3** to Memory [2 + R10 + 1]
LDW R8 0(R10)	22240000	Load double word from memory [0 + R10] into **R8 &** Load double word from memory [0 + R10 + 1] into **R9**

LDW R10 2(R11)	22AC0002	Load double word from memory [2 + R11] into **R10 &** Load double word from memory [2 + R11+ 1] into **R11**

Table 8: Simulation Test 3 Instructions Table

Expected Value:

Location	Expected Value
R0	1220
R1	214
R2	1220
R3	50
R8	1220
R9	214
R10	1220
R11	50
Memory [0]	1220
Memory [1]	214
Memory [2]	1220
Memory [3]	50

Table 9:Expected Result For Simulation Test 3

Simulation Values:

```
[Time 155000] Register File:
  ----------------------------------------
  | R0 =1220 | R7  =    0 |
  | R1 = 214 | R8  =1220 |
  | R2 =1220 | R9  = 214 |
  | R3 =  50 | R10 =1220 |
  | R4 =   0 | R11 =  50 |
  | R5 =   0 | R12 =   0 |
  | R6 =   0 | R13 =   0 |
  ----------------------------------------
  | R14 (RA) =    0      |
  ----------------------------------------
```

```
Time: 105000 | MemRd: D=1, ID
WROTE 50 TO MEM[3]
M[0] = 1220
M[1] = 214
M[2] = 1220
M[3] = 50
M[4] = 0
M[5] = 0
M[6] = 0
M[7] = 0
M[8] = 0
M[9] = 0
M[10] = x
READ from MEM[0] = 1220
```

Figure 31:Actual Simulation Test 3 Values

Waveform Generation:

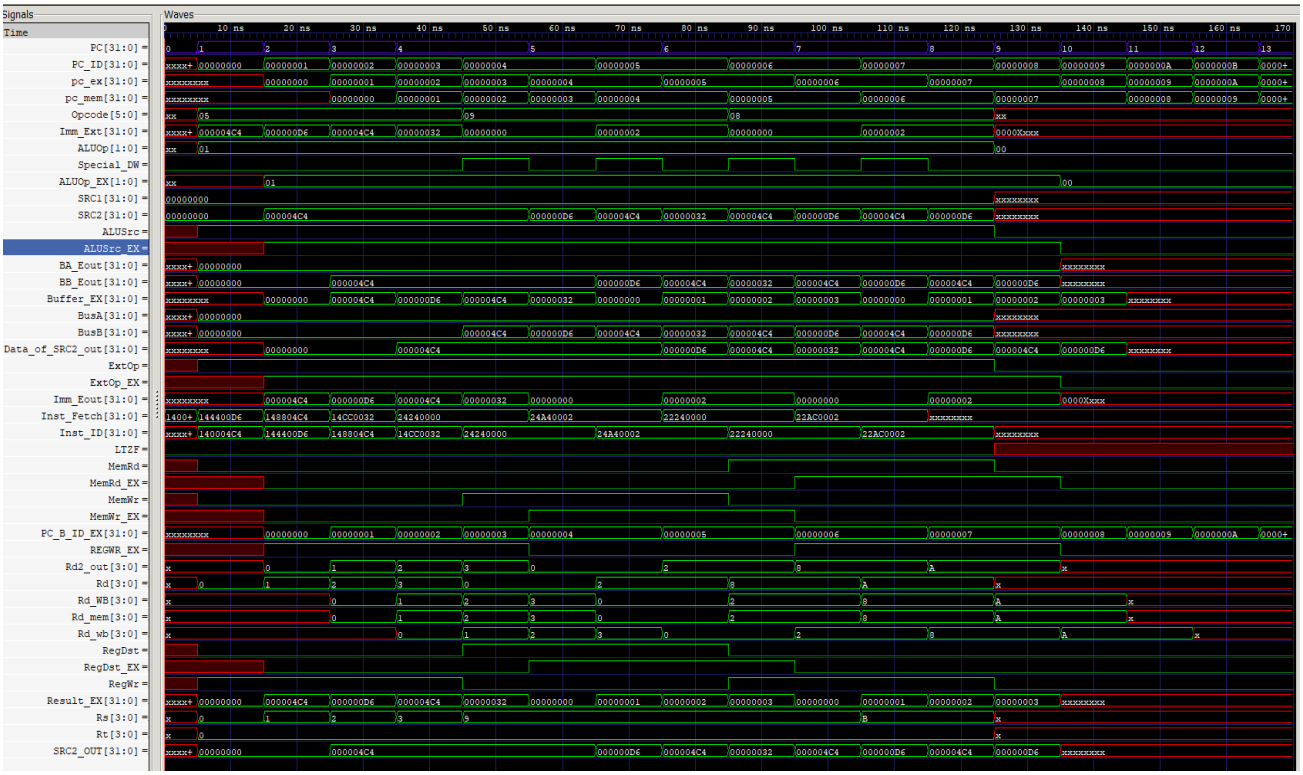


Figure 32:Waveform for Simulation Test 3

Teamwork

We ensured equal contribution to all aspects of the project, to **Datapath design, Code implementation, Simulation, and writing the report**. Each of us took the lead in specific areas to streamline our workflow but in the end the work was divided equally for all areas, we collaborated through live share extension for Verilog and OneDrive for Datapath and report.

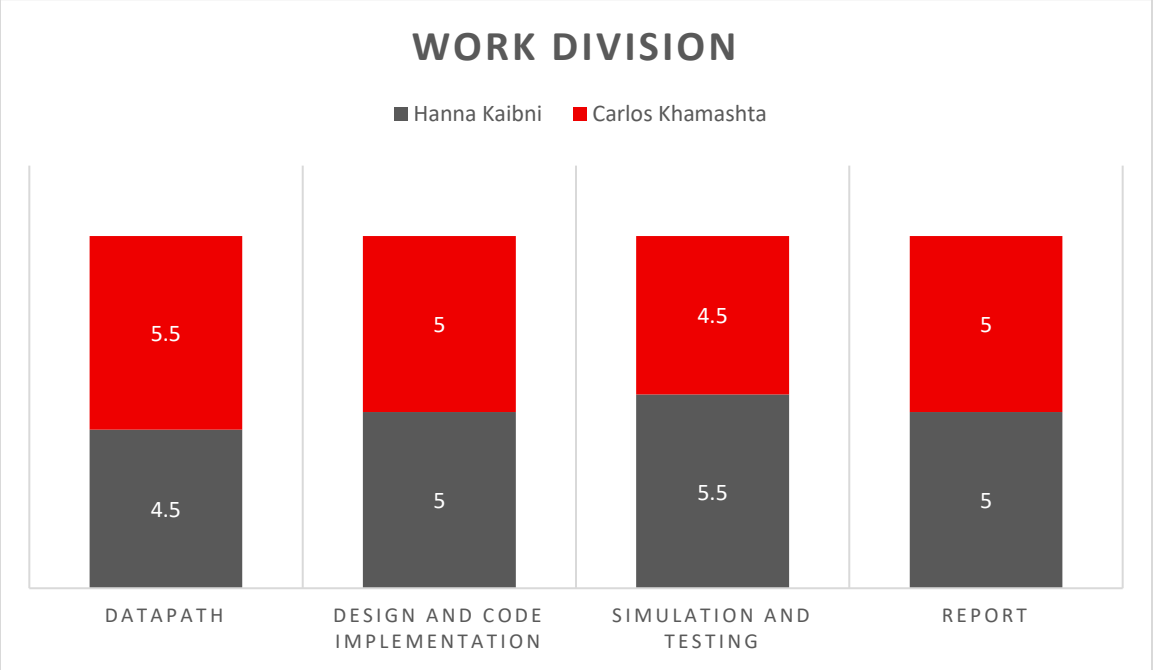


Figure 33: Teamwork Distribution

Overall 50%.

Conclusion

This project shows the successful design of a five-stage pipelined RISC processor using VS-Code Running Icarus Verilog and GTKWAVE. The processor was built in separate parts (modules) to keep the design clear, organized, and fast. It supports all main instruction types, like math, logic, memory access, and control-flow, and even includes some extra (pseudo) instructions. The Datapath and control parts were carefully set up to handle problems like hazards and keep the pipeline running correctly. After testing and simulation, the processor was shown to run different instruction sequences accurately and efficiently. This project shows a strong understanding of how pipelined processors work and creates a solid base for future improvements or added features.